

Control Hijacking: Defenses

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Memory Corruption Attacks

- The goal of attackers who attempt to corrupt the memory of a process **usually** is to **take control of it**
 - Run with the privilege of the process
 - Execute attacker-specified code
- Control-hijacking attacks are made possible by
 - Mixing **data** and **control (code)** in memory
 - Allowing a **user** to **overwrite control data**
 - Usually a pointer for future code execution

Reasoning about Memory Safety

- Memory Safety
 - No access to “undefined” memory
 - “Undefined” is with respect to the semantics of the programming language
 - Accesses may include read, write and execute
- How can we have confidence that the code we write executes in a safe fashion?
 - Build confidence by a module-by-module analysis

Reasoning about Memory Safety (Cont.)

- Reason about the memory properties of using a function or executing a statement
 - **Preconditions**: what must hold for the function or the statement to operate correctly
 - **Postconditions**: what holds after the function returns or the statement is executed
 - The postcondition of Stmt#1 should logically imply the precondition of Stmt#2
 - Invariants: the conditions that always hold at a given point in a function

Preconditions & Postconditions

- **Precondition**

- An assertion that must hold about the inputs of a function $f()$ to make it work correctly
- The caller must meet the precondition
- The callee $f()$ may assume that the precondition is met

- **Postcondition**

- An assertion that holds after a function $f()$ returns
- The caller may assume that the postcondition holds
- The callee must make sure that the postcondition holds

Reasoning about Memory Safety (Cont.)

```
/* requires: p != NULL (and p is a valid pointer) */
```

```
int deref(int *p) {  
    return *p;  
}
```

- **Precondition:** what must hold for the function to operate correctly
 - Needs to be written in a way that a person calling the function knows how to evaluate

Reasoning about Memory Safety (Cont.)

```
/* eusures: retval != NULL (and is a valid pointer) */
```

```
void *my_malloc(size_t n) {  
    void *p = malloc(n);  
    if (!p) { perror("malloc"); exit(1); }  
    return p;  
}
```

- **Postcondition:** what the function promises to hold after it returns
 - Also needs to be written in a way that a person using the function knows how to make use of the function

Reasoning about Memory Safety (Cont.)

```
int count(int a[], size_t n, int val) {  
    int c = 0;  
    for (size_t i=0; i<n; i++) {  
        /* requires a != NULL && 0 <= i && i < size(a) */  
        if (a[i] == val) c++;  
    }  
    return c;  
}
```

We assume a never changes

- General correctness proof strategy for memory safety
 - Identify each memory access code point
 - Write down precondition it requires
 - Propagate requirement up to the beginning of the function

Reasoning about Memory Safety (Cont.)

```
/* requires a != NULL */
int count(int a[], size_t n, int val) {
    int c = 0;
    for (size_t i=0; i<n; i++) {
        /* requires 0 <= i && i < size(a) */
        if (a[i] == val) c++;
    }
    return c;
}
```

0 <= i ?

- General correctness proof strategy for memory safety
 - Identify each memory access code point
 - Write down precondition it requires
 - Propagate requirement up to the beginning of the function

Reasoning about Memory Safety (Cont.)

```
/* requires a != NULL && n <= size(a) */
int count(int a[], size_t n, int val) {
    int c = 0;
    for (size_t i=0; i<n; i++) {
        /* invariant? i < n && n <= size(a) */
        /* requires i < size(a) */
        if (a[i] == val) c++;
    }
    return c;
}
```

$i < n$

$i < \text{size}(a)$?

n never changes

/* invariant: $a \neq \text{NULL} \ \&\& \ 0 \leq i \ \&\& \ i < n \ \&\& \ n \leq \text{size}(a)$ */

- A more complicated loop might require us to use induction
 - Base case: first entrance into loop
 - Induction: show that postcondition of the last statement in loop plus loop test condition, implies invariant

Class Practice

- What is the post condition of this function?

```
/* eusures: ??? */
```

```
int hash(char *s) {  
    int h = 23;  
    while (*s)  
        h = 251*h + (*s++) + 3;  
    return h % N;  
}
```

Memory Corruption Defenses

- Prevention
 - Write bug-free program (can be difficult)
 - Rewrite program in a memory-safe language that automatically checks boundary and does not allow pointer arithmetic
 - Analyze the program for potential vulnerabilities before execution
 - Prevent attack code execution
 - Make exploitation harder

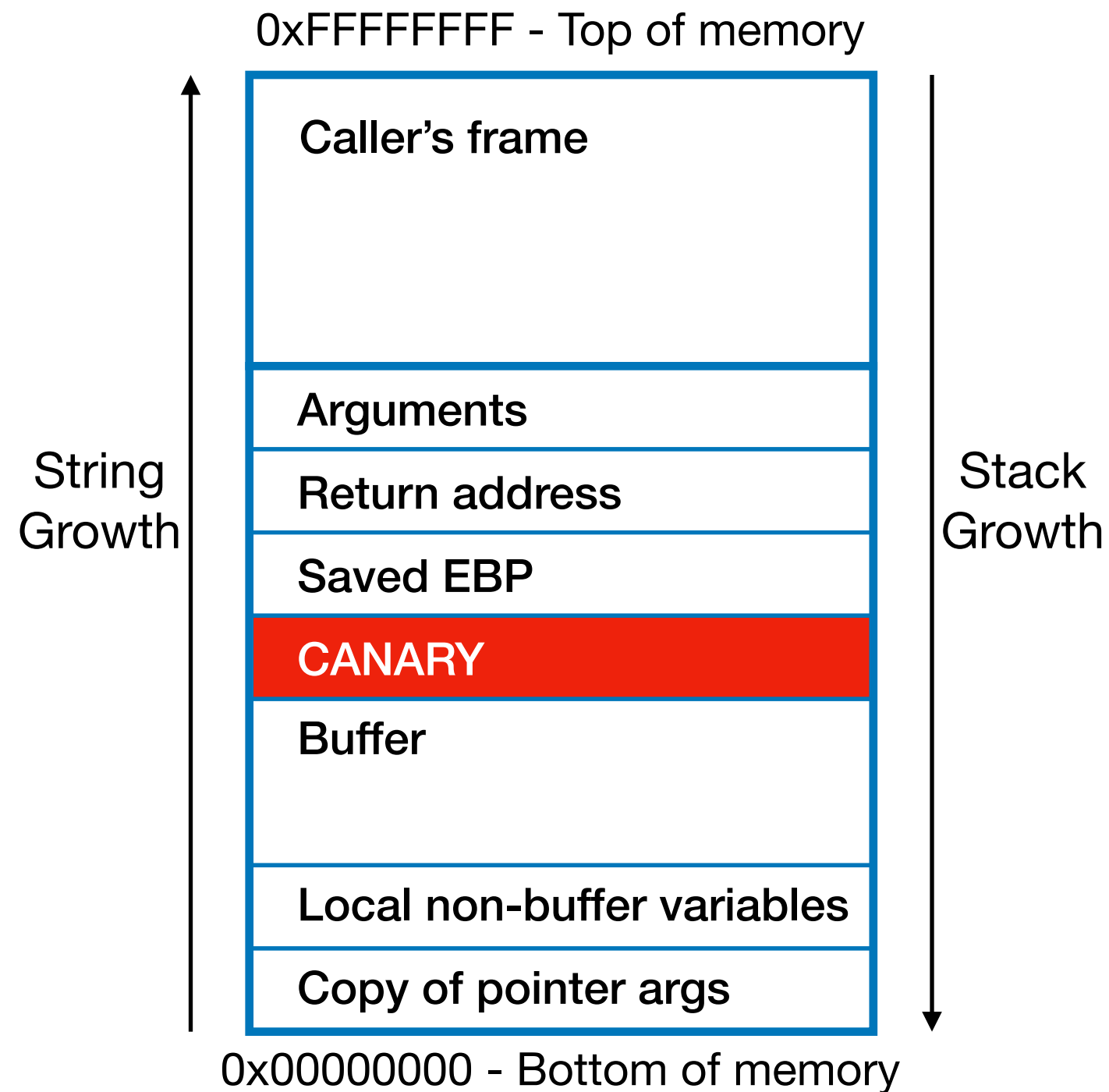
Memory Corruption Defenses (Cont.)

- Detection
 - Perform security checks during the execution of the program
 - Detect abnormal sequences of executed instructions
 - Check for integrity
- Many defense mechanisms try to prevent memory errors from being **exploitable for code injection**
 - Some defense transforms control-hijacking into denial of service

Defense #1: Canaries

- By performing run time checks for stack integrity, we can harden the executable
- **Canaries** are placed in stack frames to prevent/detect the overwrite of the return address on the stack
 - StackGuard, USENIX Security 1998
- **StackGuard** writes a canary before the return address in **each stack frame**
- When the function is returned, the canary value is verified against the stored value
- Minimal performance overhead: 8% for Apache

Stack Protection Implementations



Canary Types

- Terminator Canary
 - NULL (0x00), CR (0x0d), LF (0x0a), EOF (0xff)
 - String functions will not copy beyond a terminator
 - Attackers cannot use string functions to overwrite the return address
- Random canary
 - Random value chosen at program startup
 - Stored in location known only to the validation code and protected with unmapped pages
- XOR canary
 - Random value $\wedge$ return address

Canary Entropy

- On 32-bit x86 machines, the canary is a 32-bit value
 - 64-bit value on x86-64
- For terminator canaries, they are not completely random
 - For example, there are always 4-bit zeros
 - This makes guessing the canary much easier
 - It might be brute forced

Limitations of Canaries

- Canaries do not provide full protection
 - Canary can be learned
 - Format string vulnerabilities
 - Information leakage
 - Then it can be overwritten
 - Other pointers can still be overwritten
 - A function pointer in the function frame
 - This requires knowledge of the process memory layout
 - The **VTable** of a C++ object on the stack
 - Heap overflow

Limitations of Canaries (Cont.)

- Canaries can be bypassed by overflowing a pointer used as a destination of a strcpy() like function
- Some stack smashing techniques can overwrite the return address without touching the canaries
 - The XOR canary was introduced to protect from this attack
- Enabling canary protection requires recompile the program
 - StackGuard is implemented as a GCC patch

Defense #2: Non-Executable Memory

- In order to prevent the attack code from executing, the OS can mark the stack and the heap as **non-executable**
- Linux leverages the **NX (no-execute) bit** in the page table entry to mark certain memory areas as non-executable
 - AMD: NX bit; Intel: XD bit
 - The kernel needs to use the Physical Address Extension mode
- Windows implements the NX mechanism using **Data Execution Prevention (DEP)**
 - It can emulate the mechanism if the NX bit is not supported by the processor
- OpenBSD implements **W^X (W xor X)**
 - It forces a page to be either writable or executable but not both

Non-Executable Memory (Cont.)

- These mechanisms aim to **never** have a memory area that is both writable and executable
- Some applications may require a writable memory area that is also executable
 - Example: JIT interpreters
- Attackers can still hijack the control flow of a vulnerable application even if the memory is non-executable
 - Return into libc
 - Return oriented programming

Control Hijacking w/o Code Injection

- An attacker cannot launch code injection attacks if the memory is non executable
- The attacker can still hijack the control of a program by leveraging existing code
 - Idea: find functions or code chunks in existing libraries and glue them together to perform the desired tasks
 - Is it possible?
 - Difficult to do on your own
 - Many tools available

Defense #3: Address Space Layout Randomization

- Map the stack, the heap, the code, and the shared libraries to **random location** in process memory
 - The libraries' code need to be position-independent
 - Otherwise, run-time mapping incurs some overhead
- Jumping to a library function is **more difficult** (**not impossible**)
- Entropy
 - Windows 7: 8 bits of entropy for DLLs (1 in 256 chances)
 - 64-bit Windows 8: 24 bits of entropy
 - 32-bit Linux: 16 bits of entropy (32K attempts in average)
 - 64-bit Linux: much higher entropy

ASLR in Linux

- ASLR is enabled in Linux by default
 - `/proc/sys/kernel/randomize_va_space`
 - Can be disabled: `setarch `uname -m` -R /bin/bash`
- ASLR is implemented by the kernel in collaboration with the ELF loader
 - Stack ASLR
 - Libs/mmap ASLR
 - Exec ASLR (Requires executables in PIE format)
 - brk ASLR
 - vDSO ASLR

Exploiting ASLR

- Brute-forcing
 - Low-entropy
 - If it is possible to limit the variation space
- Attacks
 - Address leaking
 - Example #1: Format string attack
 - Example #2: ?
 - Control-flow hijacking

Defense #4: Control Flow Integrity

- Programs have a **control-flow graph (CFG)**
- Control-hijacking attacks usually result in execution paths that **do not exist** in the CFG
- Control Flow Integrity ensures that the application control flows as specified by the CFG
- Enforcing CFI:
 - Building the CFG at **compile time**
 - Instrumenting the application to check at **run time** the validity of a control transfer

Control Flow Guard (Windows 10)

- Prevent jumping to invalid locations through **indirect calls** by checking against a bit mask of all valid function entry points in the executable
- Does not prevent an attacker from jumping to a **valid wrong** function
- Hard to statically build an accurate CFG

Why does Software Have Memory Corruption Vulnerabilities?

- Programmers make mistakes
 - They are human
 - Use tools to detect/fix mistakes
- Programmers are not security-aware
 - Learn about common security flaws
- Programming languages are not well designed for security
 - Use better languages (Go, Rust, Java, Python, ...)
 - Anything but C/C++

Finding Security Problems

- Testing the security of a software is difficult
 - We need to find the abnormal/rare cases
 - Normal inputs usually would not trigger the problems
 - Security is a negative property
- How to test for security issues?
 - Random inputs
 - Or mutate the inputs with some knowledge
- How do we know we find a security problem?
 - Program crash, or any other unusual behavior
- How do we know we find all potential problems?
 - It is very difficult, but code coverage tools can help a bit

Finding Security Problems (Cont.)

- In addition to random testing (fuzzing), we can
 - Scan for known vulnerabilities
 - Ask someone to break your systems (penetration testing)
- What to do when something wrong is found?
 - Patch your software
- Patching can be difficult/expensive
 - May break some other functionalities
 - May require a stop or restart of production systems
 - It is endless

Summary

- Memory corruption may happen if an attacker can write beyond bounds of a variable
 - Attacker may be able to hijack the control of a program
- The use of safe coding, safe languages, and existing protection mechanisms can greatly reduce the risk of exploitation
- Application-logic attacks are still possible

Credits

- Some slides are based on the slides of Giovanni Vigna, Dan Boneh, Raluca Ada Popa and Nicholas Weaver